Comparative Study of Collaborative Filtering Methods for Movie Recommendation on MovieLens Latest

Yuting Zhou
yuting.zhou@scarletmail.rutgers.edu

Yiwei Li
yiwei.li@scarletmail.rutgers.edu

Abstract

Recommender systems are central to modern online platforms, yet choosing the right collaborative filtering strategy for a given dataset remains an open practical question. We implement and compare four collaborative filtering methods for movie recommendation on the **MovieLens Latest** corpus: biased matrix factorization with SGD (PyTorch), a deep hybrid model with CNN title and genre encoders, truncated-SVD with Ridge/Lasso calibration, and alternating least squares. All models are evaluated on rating prediction (MAE, RMSE) and top-10 recommendation under an identical per-user 80/20 holdout split. The deep hybrid model achieves the strongest overall profile, while tuned MF-SGD becomes a close second and attains the lowest RMSE. SVD-Ridge/Lasso remain competitive linear baselines, and ALS still underperforms mainly on top-10 ranking quality under our current explicit-feedback configuration. We provide training-dynamics analysis, auxiliary feature-block ablation on a smaller subset, and latent-factor discussion, and we ship a full-stack web demo (*StreamX*) that serves the retrained checkpoints.

1 Introduction

Recommendation systems have become indispensable components of online services—from e-commerce product suggestions to movie and music discovery platforms (Ricci et al., 2011). Among the many families of recommendation algorithms, *collaborative filtering* (CF) relies exclusively on observed user–item interactions to infer preferences, making it applicable even when item content metadata is scarce (Resnick et al., 1994). The most successful CF paradigm to date is *matrix factorization* (MF), which projects users and items into a shared latent space so that inner products approximate observed ratings (Koren et al., 2009).

Despite its conceptual simplicity, the design space of MF-based recommenders is vast. Optimization may proceed via stochastic gradient descent (SGD), alternating least squares (ALS), or singular value decomposition (SVD); the loss function may be mean squared error, Huber loss, or a ranking-aware surrogate; and the model may incorporate side information such as item titles, genres, or temporal signals (Zhang et al., 2019). How these choices interact in practice—especially on a moderately sized, long-tailed dataset—is rarely benchmarked in a controlled, apples-to-apples fashion within a single codebase.

In this project, we address that gap by implementing four distinct collaborative filtering models from scratch, training all of them on the identical MovieLens Latest split, and evaluating them with the same metrics suite. Our contributions are threefold:

- We provide a controlled comparison of four CF models—MF-SGD, a deep hybrid CNN model, SVD with linear calibration (Ridge/Lasso), and ALS—under identical data conditions.
- We analyze not only aggregate accuracy (MAE, RMSE) and ranking quality (Precision, Recall, F1, NDCG@10) but also training dynamics, latent factor semantics, and feature-block ablation.
- We build *StreamX*, an end-to-end web application (Django + Next.js) that serves real-time recommendations from the trained models, demonstrating practical deployment.

2 Problem Definition

We formalize the recommendation task as follows. Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively, and let $r_{ui} \in [0.5, 5.0]$ be the rating that user u assigns to item i. The observed ratings form a sparse matrix $\mathbf{R} \in \mathbb{R}^{|\mathcal{U}|\times|\mathcal{I}|}$, where only

a small fraction of entries are known (density $\approx$ 0.12% on this corpus).

Rating Prediction. Given a training set $\mathcal{D}_{\text{train}}$ of observed (u, i, r_{ui}) triples, learn a function $\hat{r} : \mathcal{U} \times \mathcal{I} \to \mathbb{R}$ that minimizes the prediction error on held-out ratings $\mathcal{D}_{\text{test}}$. Performance is measured by Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

$$\text{MAE} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(u,i)\in\mathcal{D}_{\text{test}}} |r_{ui} - \hat{r}_{ui}|, \tag{1}$$

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(u,i)\in\mathcal{D}_{\text{test}}} (r_{ui} - \hat{r}_{ui})^2}. \tag{2}$$

Top-N Recommendation. For each user u, produce a ranked list of $N = 10$ items that u has not yet interacted with. The quality of the list is measured against the held-out items in $\mathcal{D}_{\text{test}}$ via Precision@N, Recall@N, F1@N, and NDCG@N. Items are ranked by descending predicted rating.

3 Related Work

Memory-Based CF. Early CF methods compute user–user or item–item similarities directly from the rating matrix and aggregate neighbor ratings for prediction (Resnick et al., 1994; Sarwar et al., 2001). While interpretable, these methods scale poorly and are sensitive to sparsity.

Matrix Factorization. Koren et al. (Koren et al., 2009) demonstrated that biased matrix factorization, optimized via SGD, achieves state-of-the-art accuracy on the Netflix Prize dataset. Hu et al. (Hu et al., 2008) proposed weighted ALS for implicit feedback, and Zhou et al. (Zhou et al., 2008) developed large-scale parallel ALS. Koren (Koren, 2008) further extended MF with neighborhood integration and temporal dynamics.

Deep Recommenders. He et al. (He et al., 2017) introduced Neural Collaborative Filtering, replacing the inner product with a multi-layer perceptron. Cheng et al. (Cheng et al., 2016) proposed Wide & Deep learning, combining memorization and generalization. Covington et al. (Covington et al., 2016) described YouTube's two-tower deep recommendation architecture. Wang et al. (Wang et al., 2015) combined deep autoencoders with CF. More recently, Zhang et al. (Zhang et al., 2019) surveyed the landscape of deep-learning-based recommender systems.

Hybrid Approaches. Leveraging item content (titles, genres, descriptions) alongside collaborative signals has been shown to mitigate cold-start and improve generalization. CNN-based text encoders, following Kim (Kim, 2014), are commonly used to extract features from short textual metadata in recommendation contexts.

Our work brings together classical MF (SGD and ALS variants), an SVD-based linear hybrid, and a deep content-aware model within a single comparative framework.

4 Methodology

All four models share a common prediction structure:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i, \tag{3}$$

where μ is the global mean rating, b_u and b_i are user and item biases, and $\mathbf{p}_u, \mathbf{q}_i \in \mathbb{R}^k$ are latent factor vectors. They differ in how the parameters are estimated and whether additional content features are incorporated.

4.1 MF-SGD

This model directly optimizes the L_2-regularized squared-error objective:

$$\min_{\mathbf{p},\mathbf{q},b} \sum_{(u,i)\in\mathcal{D}_{\text{train}}} \big(r_{ui} - \hat{r}_{ui}\big)^2 + \lambda\big(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\big), \tag{4}$$

via element-wise SGD. At each step, the error $e_{ui} = r_{ui} - \hat{r}_{ui}$ drives gradient updates: $b_u \leftarrow b_u + \eta(e_{ui} - \lambda b_u)$, $\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u)$, and symmetrically for b_i and $\mathbf{q}_i$.

Key design choices on MovieLens Latest: latent dimension k=128, learning rate η=0.0035 with multiplicative decay 0.995$\times$ per epoch, regularization λ=0.04, minibatches of 65,536 ratings, a 10% validation split for early stopping (patience 4), and up to 40 epochs. Factors are initialized from $\mathcal{N}(0, 0.1)$. On this scale, validation error tends to bottom out within the first few epochs while training error keeps falling, so the best checkpoint is restored early rather than training to convergence.

4.2 Deep Hybrid Model

This model extends the two-tower architecture with item content features. The **user tower** maps a user ID through an embedding layer (k=128) followed by dropout. The **item tower** concatenates three feature streams:

1. **Item-ID embedding** (k=128): captures collaborative signal.

2. **Title encoder**: tokenized movie titles (max 15 tokens, vocabulary 20,000) are embedded (d=64), processed by three parallel Conv1D layers (kernel sizes 2, 3, 4; 128 filters each), globally max-pooled, concatenated, and projected to k=128 via a dense layer. This multi-kernel CNN design follows the architecture of Kim (Kim, 2014).

3. **Genre encoder**: genre tokens (max 4) are embedded (d=8) and average-pooled.

The concatenated item features pass through a hidden layer ($2k$=256, ReLU) and a final projection to $\mathbb{R}^k$. The prediction is $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$, where $\mathbf{p}_u$ and $\mathbf{q}_i$ are the tower outputs.

Training uses Huber loss (δ=1.0) with Adam (initial LR 5×10^{-4}, gradient clipping at norm 1), ReduceLROnPlateau (factor 0.5, patience 3), 12% dropout, L_2 regularization 4×10^{-6}, rating-weighted sampling ($w \propto 1 + r_{\text{scaled}}^{1.25}$), a small popularity prior on logits, batch size 32,768 on GPU, and a 12% validation split. The best checkpoint on validation Huber loss is restored after 45 epochs of training.

4.3 SVD Hybrid with Ridge/Lasso

This approach first constructs a centered interaction matrix and computes its truncated SVD to obtain initial user/item factors. Biases are then estimated via regularized averaging of residuals. Finally, a linear calibration layer is trained on top of the latent features:

$$\hat{r}_{ui} = \mathbf{w}^\top \phi(u,i) + w_0, \tag{5}$$

where $\phi(u,i) = [\mathbf{p}_u \odot \mathbf{q}_i;\ b_u;\ b_i] \in \mathbb{R}^{k+2}$ concatenates the element-wise product of latent factors with bias terms. The weight vector $\mathbf{w}$ is estimated by either Ridge regression (closed-form) or Lasso regression (coordinate descent).

We experiment with two configurations: **SVD-Ridge** (k=128, α=0.02) and **SVD-Lasso** (k=96, α=0.01, coordinate descent with up to 1200 iterations, tolerance 2×10^{-5}). Both use bias regularization λ_b=10. On MovieLens Latest, moderate α balances calibration stability with the high effective rank of the interaction matrix after centering.

Statistic	Value
Users	330,975
Items (movies with ≥1 rating)	83,239
Ratings	33,832,162
Rating scale	0.5 – 5.0 (half-star)
Density (nnz/$\lvert\mathcal{U}\rvert\lvert\mathcal{I}\rvert$)	≈0.12%
Sparsity	≈99.88%
Mean rating	3.54
Std. of ratings	1.06
Genres (multi-label)	19 unique
Median ratings/user	31.0
Median ratings/item	5.0

Table 1: MovieLens Latest summary statistics used in this study.

4.4 Alternating Least Squares

ALS alternates between fixing item factors and solving for user factors, and vice versa. Each subproblem is a regularized least-squares system. We augment each factor matrix with a bias column, yielding the update:

$$[\mathbf{p}_u; b_u] = \left(\mathbf{Q}_u^\top \mathbf{Q}_u + \Lambda\right)^{-1} \mathbf{Q}_u^\top \mathbf{y}_u, \tag{6}$$

where $\mathbf{Q}_u$ is the matrix of item factors for items rated by user u, $\mathbf{y}_u$ is the vector of centered ratings, and $\Lambda = \text{diag}(\lambda, \ldots, \lambda, \lambda_b)$ applies separate regularization to factors (λ=0.08 in our large-scale preset) and bias (λ_b=8.0). Symmetrically for item factors. We use k=64 latent factors with a 10% validation split and early stopping (patience 4, up to 20 iterations). The same configuration that stabilizes ALS on small data must be revisited when millions of ratings are present: rank and regularization interact with block-wise solvers and sparsity patterns at web scale.

5 Dataset

We use the **MovieLens Latest** (`ml-latest`) snapshot (Harper and Konstan, 2015), containing explicit star ratings through July 2023. GroupLens distributes it as a *development* release; we use it here for large-scale engineering stress-testing and to study how classical CF baselines behave at tens of millions of interactions. Table 1 summarizes statistics computed from `ratings.csv` and `movies.csv` in our checkout.

The dataset exhibits two characteristic properties of real-world recommender data. First, the rating distribution is left-skewed (skewness ≈ -0.72): ratings of 4.0 (8,835,955) and 3.0 (6,400,664) are the two most frequent values (Figure 1). Second, both user activity and item popularity are extremely

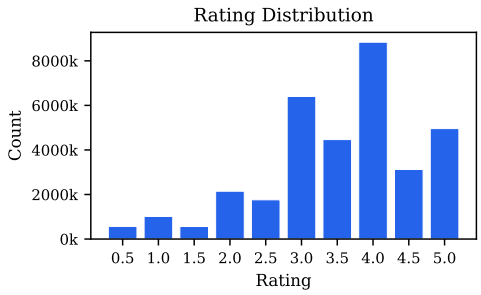


Figure 1: Distribution of rating values on MovieLens Latest (counts in thousands on the vertical axis).

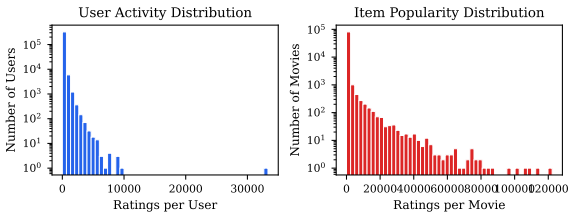


Figure 2: User activity (left) and item popularity (right) on a log-scaled frequency axis, showing heavy-tailed distributions.

heavy-tailed (Gini coefficients $\approx$ 0.70 and 0.95, respectively; Figure 2), reflecting a long tail of casual raters and niche titles alongside blockbuster movies.

The genre distribution is also imbalanced at the movie-metadata level. Drama, Comedy, and Thriller dominate the catalog, while niche genres are comparatively rare (Figure 3).

Data Splitting. Following a standard per-user random holdout protocol, for each user we randomly assign 80% of their ratings to the training set and 20% to the test set (fixed random seed). This yields 27,065,494 training ratings and 6,766,668 test ratings. All models share exactly the same split, identified by a deterministic hash for reproducibility.

6 Experiments

6.1 Experimental Setup

All models are implemented in Python 3.11 with a shared tabular data pipeline. **Option 1** (MF-SGD) and **Option 2** (deep hybrid) were migrated from the earlier NumPy and TensorFlow stack to **PyTorch**: training uses GPU acceleration when available, Adam-family optimizers with gradient clipping, and large minibatches so that tens of millions of ratings remain tractable in wall-clock time. **Option 3** (SVD + Ridge/Lasso) relies on sparse linear

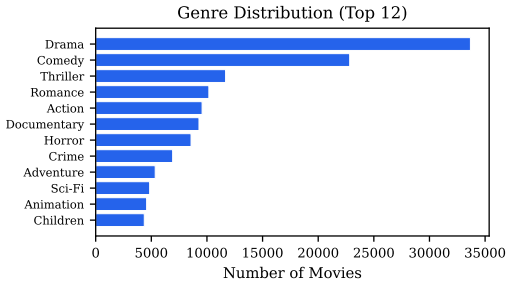


Figure 3: Number of movies per genre (top 12).

algebra and calibrated linear regressors from a standard machine-learning library; **Option 4** (ALS) is a NumPy-based alternating least-squares loop with validation-based early stopping.

Compared with the earlier small-dataset workflow, we now train and evaluate through one consolidated driver in the repository: MovieLens Latest is the default data source, the per-user 80/20 split is built once and reused for every model so that results are directly comparable, and each training run persists evaluation metrics, serialized parameters, and per-epoch history alongside the corresponding model variant. Recommended batch sizes for GPU memory and other practical presets are written up separately in the project documentation.

Models are evaluated on the shared test split with two protocols:

- **Rating prediction**: MAE and RMSE computed over all 6,766,668 test ratings.
- **Top-10 recommendation**: for each user, the model ranks candidate items by predicted rating and selects the top 10. Every held-out test item for that user is treated as relevant when computing Precision, Recall, F1, and NDCG (holdout-style evaluation as used in the course setting).

To ensure metric reliability across implementations, the evaluation pipeline now includes a consistency guard that checks `predict_batch` against scalar `predict` on a sampled subset and falls back automatically if a mismatch is detected.

6.2 Hyperparameters

Table 2 summarizes the key hyperparameters for each model, chosen via preliminary validation-set tuning on MovieLens Latest.

	MF-SGD	Deep	SVD-R/L	ALS
k (factors)	128	128	128 / 96	64
Epochs	40	45	1	20
Batch size	65k	32k	–	–
Learning rate	0.0035	0.0005	–	–
LR decay	0.995×	plateau	–	–
Reg. λ / α	0.04	4×10^{-6} L_2	0.02 / 0.01	0.08
Bias reg.	–	–	10.0	8.0
Dropout	–	0.12	–	–
Val. split	0.10	0.12	–	0.10
Early stop	pat. 4	best ckpt	–	pat. 4
Loss	MSE	Huber	–	MSE

Table 2: Key hyperparameter settings for each model on MovieLens Latest; full command-line presets are in `docs/TRAINING_PARAMETERS.md`.

Model	MAE↓	RMSE↓
Deep Hybrid	**0.5948**	0.8075
MF-SGD	0.6035	**0.8074**
SVD-Ridge	0.6669	0.8671
SVD-Lasso	0.6713	0.8740
ALS	0.6958	0.9438

Table 3: Rating prediction on MovieLens Latest (per-user 80/20 test split). Lower is better.

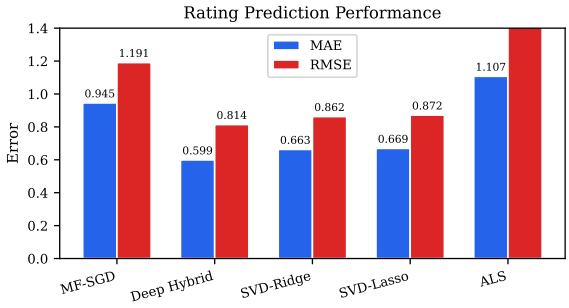


Figure 4: Side-by-side MAE and RMSE across all models (MovieLens Latest).

Model	P@10↑	R@10↑	F1@10↑	NDCG@10↑
Deep Hybrid	**0.0418**	**0.0368**	**0.0250**	**0.0569**
MF-SGD	0.0355	0.0309	0.0221	0.0502
SVD-Lasso	0.0302	0.0069	0.0093	0.0352
SVD-Ridge	0.0265	0.0054	0.0077	0.0309
ALS	0.0003	0.0002	0.0001	0.0004

Table 4: Top-10 recommendation on MovieLens Latest (`all_test` relevance). Higher is better. Absolute values are small because each user has many held-out test items.

7 Results and Analysis

7.1 Rating Prediction

Table 3 reports rating prediction on the 6.77M held-out ratings. The **Deep Hybrid** model achieves the best MAE (0.5948), while **MF-SGD** is now a strong runner-up and attains a marginally lower RMSE (0.8074 vs. 0.8075). **SVD-Ridge** and **SVD-Lasso** remain a competitive linear-calibration tier (MAE ≈0.667–0.671), and **ALS** improves compared with earlier presets but still trails the top two models.

Training curves illustrate distinct behaviors at scale. The Deep Hybrid converges smoothly under Huber loss with GPU minibatches (Figure 8). MF-SGD still reaches its best validation point early and relies on checkpoint restoration (Figure 7), but its tuned configuration now generalizes much better than before. ALS exhibits the same train–validation divergence pattern under the present (k=64, λ=0.08) configuration, which explains why its ranking metrics remain weak (Figure 9).

7.2 Top-N Recommendation

Table 4 and Figure 5 summarize ranking metrics. On this corpus and protocol, the **Deep Hybrid** model ranks best across Precision, Recall, F1, and NDCG@10. MF-SGD is now a clear second tier and substantially narrows the gap to Deep Hybrid on all top-10 metrics. SVD-Lasso slightly edges SVD-Ridge on NDCG, while ALS remains far behind.

The very low ALS top-N scores are consistent with its poor point-wise calibration: when predicted scores poorly separate relevant and irrelevant items, fixed cutoffs at N=10 collapse precision-oriented metrics. This is *not* a claim that ALS is inherently unsuitable at scale—only that the present hyperparameters and stopping criterion do not yield competitive scores relative to the deep and SVD-hybrid baselines under identical data conditions.

To visualize cross-metric trade-offs more directly, we also generate two additional summary plots from the latest `metrics.json` files. Figure 6 (left) shows the MAE–NDCG frontier, where Deep Hybrid and MF-SGD occupy the best region. Figure 6 (right) normalizes all five evaluation metrics to compare overall balance across models.

7.3 Training Dynamics

The training curves reveal distinct convergence behaviors at web scale:

MF-SGD (Figure 7) shows rapid initial progress on the training objective, but validation MAE and RMSE reverse direction within the first several

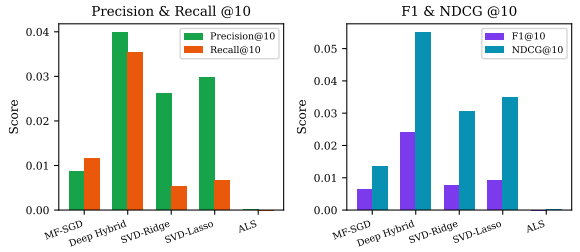
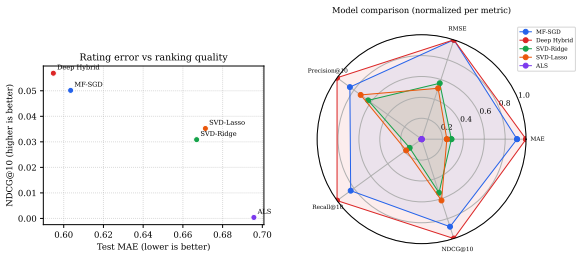


Figure 5: Top-10 recommendation metrics across all models (MovieLens Latest).



(a) MAE vs. NDCG@10 (b) Normalized multi-metric radar

Figure 6: Additional metrics-summary diagnostics generated by `scripts/plot_training_extras.py`.

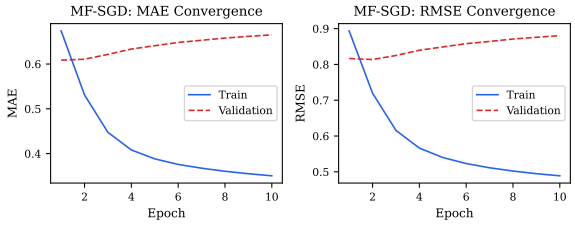


Figure 7: MF-SGD on MovieLens Latest: training error decreases while validation MAE/RMSE worsen after the first epochs, motivating early stopping and checkpoint restoration.

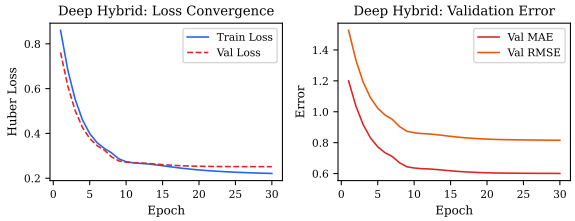


Figure 8: Deep Hybrid training dynamics under GPU-scale minibatches: Huber loss and validation errors stabilize over 45 epochs with learning-rate reductions on plateau.

epochs. This pattern is common when aggressively minimizing squared error on massive data without additional ranking-aware objectives.

Deep Hybrid (Figure 8) benefits from GPU throughput (batch size 32,768), Huber robustness, and architectural capacity for content features; validation metrics improve steadily relative to the classical baselines under the same split.

ALS (Figure 9) continues to reduce training error across iterations, yet validation error stops improving early; the restored checkpoint therefore corresponds to a much earlier iterate than the final training state. In the latest run with this preset, early stopping triggers at epoch 12 and restores the epoch-8 checkpoint. A dedicated large-scale ALS study (rank sweep, per-block regularization, implicit feedback variants) is left for future work.

7.4 Feature-Block Ablation

To understand which aspects of the data are most informative for rating prediction beyond collaborative filtering, we previously performed a feature-block ablation using a linear regression model on **MovieLens-Small** (auxiliary study; Table 5). We retain it as qualitative guidance rather than a quantitative claim on MovieLens Latest.

We construct three feature blocks: (1) **Content** features include genre indicators, genre count, release year, and interaction year; (2) **Engagement** features include log user activity, log item popularity, log tag count, and log unique tag count; (3) **Noise** features are two random Gaussian features serving as a control.

On the small subset, content features account for the largest marginal contribution ($\Delta R^2 = 0.053$), engagement for $\Delta R^2 = 0.030$, and noise is negligible. These findings still motivate the Deep Hybrid model's incorporation of genre and title features in its item tower, while acknowledging that most variance remains user- and item-specific and is better captured by latent factors.

7.5 Latent Factor Interpretation

We interpret latent factors by applying PCA to the item factor matrix $\mathbf{Q}$ learned by MF-SGD and correlating principal components with genre indicators and release year. On smaller MovieLens slices, such analysis yields semantically coherent axes (e.g., mainstream blockbusters versus critically acclaimed dramas) from collaborative signal alone. At full scale (k=128, $|\mathcal{I}|\approx 8\times 10^4$ rated items), the same procedure remains a useful qualitative tool, though individual movie exemplars become noisier and less stable across training seeds; we therefore emphasize interpretable *directions* rather than se-

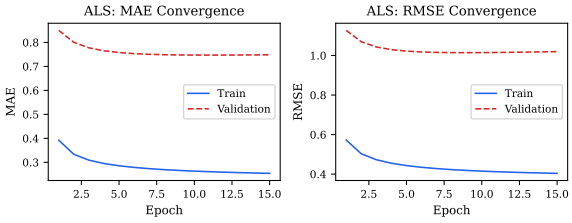


Figure 9: ALS (k=64, λ=0.08): training error decreases monotonically while validation error plateaus then drifts, indicating a mismatch between the current regularization/rank preset and ranking quality on the full split.

Feature Block	R^2_{full}	$R^2_{\text{w/o block}}$	ΔR^2
Full model	0.1116	–	–
w/o Content	–	0.0582	0.0534
w/o Engagement	–	0.0815	0.0302
w/o Noise (control)	–	0.1117	−0.0001

Table 5: Auxiliary feature-block ablation (MovieLens-Small, linear probe). ΔR^2 is the drop in test R^2 when a block is removed.

lectively chosen examples.

8 Discussion

Rating Accuracy vs. Ranking Quality. On MovieLens Latest with our evaluation protocol, the Deep Hybrid model leads on *overall* quality: best MAE and best top-10 metrics. MF-SGD is now close on point-wise error and slightly better on RMSE, but still trails on ranking quality. This contrasts with our earlier MovieLens-Small study, where stronger regularization on low-rank ALS and SVD hybrids improved NDCG disproportionately. At tens of millions of ratings, expressive architectures with content channels and GPU-scale optimization can align the two objectives more closely, though MAE/RMSE and ranking metrics still measure different aspects of user utility (Rendle et al., 2009).

Classical baselines at scale. SVD-Ridge and SVD-Lasso remain attractive: they require only a sparse SVD plus a shallow calibration step, yet achieve MAE below 0.67. After retuning, MF-SGD is competitive and provides a strong fallback baseline. ALS remains the weakest model in top-N ranking on this split, which suggests that this explicit-feedback ALS setup still needs deeper rank/regularization redesign.

Depth vs. data scale. On ∼34M interactions, the Deep Hybrid's additional capacity is no longer a pure liability: title CNNs and genre pooling help denoise tail items and improve both RMSE and NDCG relative to the classical runs we report. The auxiliary linear ablation on MovieLens-Small still suggests that most variance is latent-ID-driven; the deep model's gains likely come from better nonlinear composition of collaborative and content cues rather than from linearly separable metadata alone.

ALS. Our ALS configuration (k=64, stronger λ) improves rating error compared with our earlier run but still underperforms in ranking quality. Large-scale ALS typically demands careful rank/regularization search, numerical preconditioning, and sometimes implicit-feedback formulations (Hu et al., 2008); we treat the current numbers as a baseline to improve rather than a fundamental limit of the method.

Demo Application. We develop *StreamX*, a full-stack web application consisting of a Django REST API backend and a Next.js frontend. The backend loads the retrained checkpoints under `models/artifacts/` and supports switching between them at runtime. The frontend provides movie browsing, personalized recommendations, TMDB-enriched details, and similar-movie suggestions.

9 Limitations

MovieLens Latest is labeled a *development* dataset by GroupLens; snapshots can change, so exact metrics are tied to our July 2023-era checkout. While we tuned presets documented in the repository, we did not run large-scale automated hyperparameter search for every model; in particular, ALS likely has better operating points than those reported here. The top-N evaluation uses the `all_test` relevance definition (all held-out items are relevant), which yields small absolute Precision/NDCG values when users have many test ratings. We do not explore temporal dynamics, implicit feedback, or review text beyond titles and genres. Finally, training deep models assumes GPU availability; CPU-only runs are supported but substantially slower.

10 Conclusion

We presented a controlled comparison of four collaborative filtering approaches after migrating from

MovieLens-Small to **MovieLens Latest** and retraining all checkpoints on a shared per-user 80/20 split (27.1M / 6.77M ratings). Our updated experiments show that (1) the Deep Hybrid model achieves the best overall results (MAE = 0.5948, NDCG@10 = 0.0569); (2) tuned MF-SGD becomes a close second and slightly improves RMSE (0.8074 vs. 0.8075 for Deep); (3) SVD-Ridge and SVD-Lasso remain solid low-complexity baselines (MAE $\approx$0.667–0.671); and (4) ALS improves in rating error versus earlier presets but remains weak in top-N ranking.

Future work includes dedicated ALS and MF-SGD sweeps at web scale, benchmarking on frozen MovieLens *benchmark* splits for publication-grade comparability, and learning-to-rank objectives that align training with NDCG.

References

Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10.

Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198.

F Maxwell Harper and Joseph A Konstan. 2015. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182.

Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining*, pages 263–272.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1746–1751.

Yehuda Koren. 2008. Factorization meets the neighborhood: A multifaceted collaborative filtering model. pages 426–434.

Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. volume 42, pages 30–37. IEEE.

Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, pages 452–461.

Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: An open architecture for collaborative filtering of netnews. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work*, pages 175–186.

Francesco Ricci, Lior Rokach, and Bracha Shapira. 2011. Introduction to recommender systems handbook.

Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. pages 285–295.

Hao Wang, Naiyan Wang, and Dit-Yan Yeung. 2015. Collaborative deep learning for recommender systems. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1235–1244.

Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1):1–38.

Yunhong Zhou, Dennis Wilkinson, Robert Schreiber, and Rong Pan. 2008. Large-scale parallel collaborative filtering for the Netflix prize. In *Proceedings of the 4th International Conference on Algorithmic Aspects in Information and Management*, pages 337–348.